

MySQL

Complete Practical Reference

From Basics to Advanced — Hands-On Learning

25 Chapters • Theory + Commands + Practice Tasks

TABLE OF CONTENTS

[Chapter 1 — Database Commands](#)

[Chapter 2 — Table Commands](#)

[Chapter 3 — Constraints](#)

[Chapter 4 — CRUD Operations](#)

[Chapter 5 — Functions](#)

[Chapter 6 — GROUP BY / HAVING](#)

[Chapter 7 — Joins](#)

[Chapter 8 — Subqueries](#)

[Chapter 9 — Views](#)

[Chapter 10 — Indexes](#)

[Chapter 11 — Authorization & Privileges](#)

[Chapter 12 — Roles](#)

[Chapter 13 — Transactions](#)

[Chapter 14 — Stored Procedures](#)

[Chapter 15 — Functions \(User-Defined\)](#)

[Chapter 16 — Triggers](#)

[Chapter 17 — Events](#)

[Chapter 18 — CTE \(Common Table Expressions\)](#)

[Chapter 19 — Window Functions](#)

[Chapter 20 — JSON in MySQL](#)

[Chapter 21 — Query Optimization](#)

[Chapter 22 — Partitioning](#)

[Chapter 23 — Security](#)

[Chapter 24 — Backup & Restore](#)

[Chapter 25 — Replication](#)

Chapter 1 — Database Commands

A database is a structured collection of data. In MySQL, you manage databases using DDL (Data Definition Language) commands. Every MySQL project starts with creating and selecting a database.

Core Database Commands

Show All Databases

```
SHOW DATABASES;
```

Create a Database

```
CREATE DATABASE school;
```

```
CREATE DATABASE IF NOT EXISTS school;
```

Select / Use a Database

```
USE school;
```

Drop a Database

```
DROP DATABASE school;
```

```
DROP DATABASE IF EXISTS school;
```

Show Current Database

```
SELECT DATABASE();
```

Show Database Creation SQL

```
SHOW CREATE DATABASE school;
```



TIP

Always use IF NOT EXISTS / IF EXISTS to avoid errors in scripts.

Practice Tasks

Task 1	Create a database called company.
Task 2	Create a database called test_db only if it doesn't already exist.
Task 3	List all databases on your server.
Task 4	Switch to the company database.
Task 5	Drop the test_db database safely.

Chapter 2 — Table Commands

Tables are the core storage units in a relational database. DDL commands let you create, modify, rename, copy, and drop tables. Understanding data types is essential for good schema design.

Common Data Types

```
-- Integer types
INT, TINYINT, SMALLINT, BIGINT
-- String types
VARCHAR(n), CHAR(n), TEXT, MEDIUMTEXT, LONGTEXT
-- Numeric
DECIMAL(p,s), FLOAT, DOUBLE
-- Date / Time
DATE, TIME, DATETIME, TIMESTAMP, YEAR
-- Boolean
BOOLEAN -- stored as TINYINT(1)
-- JSON
JSON
```

Table Commands

Create Table

```
CREATE TABLE employees (
  emp_id    INT AUTO_INCREMENT PRIMARY KEY,
  name      VARCHAR(100) NOT NULL,
  dept      VARCHAR(50),
  salary    DECIMAL(10,2) DEFAULT 0.00,
  joined    DATE
);
```

Show Tables

```
SHOW TABLES;
SHOW TABLES LIKE 'emp%';
```

Describe Table Structure

```
DESC employees;
DESCRIBE employees;
SHOW COLUMNS FROM employees;
```

Show Create Table SQL

```
SHOW CREATE TABLE employees;
```

Rename Table

```
RENAME TABLE employees TO staff;
ALTER TABLE staff RENAME TO employees;
```

Copy Table

```
-- Copy structure + data
CREATE TABLE employees_backup AS SELECT * FROM employees;
-- Copy structure only
CREATE TABLE employees_empty LIKE employees;
```

Modify / Alter Table

```
-- Add a column
ALTER TABLE employees ADD COLUMN phone VARCHAR(15);
-- Drop a column
ALTER TABLE employees DROP COLUMN phone;
-- Modify column type
ALTER TABLE employees MODIFY COLUMN name VARCHAR(200) NOT NULL;
-- Rename column
ALTER TABLE employees CHANGE name full_name VARCHAR(200);
-- Add column at specific position
ALTER TABLE employees ADD COLUMN age INT AFTER name;
```

Truncate vs Drop

```
TRUNCATE TABLE employees; -- Removes all rows, keeps structure
DROP TABLE employees;     -- Removes table entirely
DROP TABLE IF EXISTS employees;
```

NOTE

TRUNCATE is faster than DELETE for clearing all rows — it does not fire triggers.

Practice Tasks

Task 1	Create a table called products with: id (PK, auto-increment), name, price, stock, created_at.
Task 2	Add a column description TEXT to the products table.
Task 3	Rename the column stock to quantity.
Task 4	Create a backup of products table with all data.
Task 5	Truncate the products_backup table.

Chapter 3 — Constraints

Constraints enforce rules at the database level, ensuring data integrity. They prevent bad data from being inserted and maintain relationships between tables.

Types of Constraints

PRIMARY KEY	-- Uniquely identifies each row, NOT NULL + UNIQUE
FOREIGN KEY	-- Links two tables, enforces referential integrity
UNIQUE	-- No duplicate values allowed
NOT NULL	-- Column cannot be empty
DEFAULT	-- Provides default value if none given
CHECK	-- Validates value against a condition (MySQL 8.0+)
AUTO_INCREMENT	-- Auto-generates sequential integers

Constraint Examples

PRIMARY KEY

```
-- Inline
CREATE TABLE orders (
  order_id INT PRIMARY KEY AUTO_INCREMENT,
  ...
);

-- Table-level
CREATE TABLE order_items (
  order_id INT,
  product_id INT,
  PRIMARY KEY (order_id, product_id) -- Composite PK
);
```

FOREIGN KEY

```
CREATE TABLE orders (
  order_id INT AUTO_INCREMENT PRIMARY KEY,
  customer_id INT,
  FOREIGN KEY (customer_id) REFERENCES customers(id)
  ON DELETE CASCADE
  ON UPDATE CASCADE
);
```

ON DELETE / ON UPDATE Actions:

CASCADE	-- Propagate delete/update to child rows
SET NULL	-- Set FK column to NULL when parent deleted
RESTRICT	-- Prevent delete/update if child rows exist
NO ACTION	-- Same as RESTRICT (default)

UNIQUE, NOT NULL, DEFAULT, CHECK

```
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
```

```

email    VARCHAR(200) NOT NULL UNIQUE,
age      INT CHECK (age >= 18),
status   VARCHAR(20) DEFAULT 'active',
username VARCHAR(50) NOT NULL
);

```

Adding Constraints After Creation

```

ALTER TABLE users ADD CONSTRAINT uq_email UNIQUE (email);
ALTER TABLE orders ADD CONSTRAINT fk_cust
    FOREIGN KEY (customer_id) REFERENCES customers(id);
ALTER TABLE users ADD CONSTRAINT chk_age CHECK (age >= 0);

```

Drop a Constraint

```

ALTER TABLE orders DROP FOREIGN KEY fk_cust;
ALTER TABLE users DROP INDEX uq_email; -- Drop UNIQUE
ALTER TABLE users DROP CHECK chk_age;

```

NOTE

CHECK constraints were silently ignored before MySQL 8.0.16. Always use MySQL 8+.

Practice Tasks

Task 1	Create a customers table with id, name, email (unique, not null), age (check age >= 0).
Task 2	Create an orders table with a FK to customers that cascades on delete.
Task 3	Add a default value of 'pending' to the orders status column.
Task 4	Add a composite primary key to an order_items table.
Task 5	Drop the foreign key from orders table and re-add with SET NULL on delete.

Chapter 4 — CRUD Operations

CRUD stands for Create, Read, Update, Delete — the four fundamental database operations. These are DML (Data Manipulation Language) commands you will use constantly in every project.

INSERT — Create

```
-- Single row
INSERT INTO employees (name, dept, salary, joined)
VALUES ('Alice', 'IT', 75000, '2023-01-15');

-- Multiple rows
INSERT INTO employees (name, dept, salary) VALUES
    ('Bob', 'HR', 60000),
    ('Charlie', 'IT', 80000),
    ('Diana', 'Finance', 70000);

-- Insert from SELECT
INSERT INTO employees_backup SELECT * FROM employees;

-- INSERT IGNORE (skip on duplicate key error)
INSERT IGNORE INTO employees (name, dept) VALUES ('Alice', 'IT');

-- ON DUPLICATE KEY UPDATE (upsert)
INSERT INTO employees (id, name, salary)
VALUES (1, 'Alice', 80000)
ON DUPLICATE KEY UPDATE salary = 80000;
```

SELECT — Read

```
-- All columns
SELECT * FROM employees;

-- Specific columns
SELECT name, salary FROM employees;

-- Aliases
SELECT name AS employee_name, salary * 12 AS annual_salary
FROM employees;

-- Filtering with WHERE
SELECT * FROM employees WHERE dept = 'IT';
SELECT * FROM employees WHERE salary > 60000 AND dept = 'IT';
SELECT * FROM employees WHERE dept IN ('IT', 'HR');
SELECT * FROM employees WHERE name LIKE 'A%';
SELECT * FROM employees WHERE salary BETWEEN 60000 AND 80000;
SELECT * FROM employees WHERE joined IS NULL;

-- ORDER BY
```

```

SELECT * FROM employees ORDER BY salary DESC;
SELECT * FROM employees ORDER BY dept ASC, salary DESC;

-- LIMIT & OFFSET (pagination)
SELECT * FROM employees LIMIT 10;
SELECT * FROM employees LIMIT 10 OFFSET 20;

-- DISTINCT
SELECT DISTINCT dept FROM employees;

```

UPDATE — Modify

```

-- Update single column
UPDATE employees SET salary = 90000 WHERE emp_id = 1;

-- Update multiple columns
UPDATE employees
SET salary = 85000, dept = 'Senior IT'
WHERE name = 'Alice';

-- Update based on condition
UPDATE employees SET salary = salary * 1.10
WHERE dept = 'IT';

-- Safe mode tip: use LIMIT
UPDATE employees SET status = 'inactive' WHERE joined < '2020-01-01' LIMIT 100;

```

⚠ NOTE
Always use WHERE with UPDATE! Without it, ALL rows get updated.

DELETE — Remove

```

-- Delete specific row
DELETE FROM employees WHERE emp_id = 5;

-- Delete with condition
DELETE FROM employees WHERE dept = 'Temp';

-- Delete all rows (dangerous!)
DELETE FROM employees;

-- DELETE with LIMIT
DELETE FROM employees WHERE status = 'inactive' LIMIT 50;

```

i INFO
TRUNCATE TABLE is faster for clearing all rows but can't be rolled back in some configurations.

Practice Tasks

Task 1	Insert 5 employees into your employees table with different departments.
Task 2	Select all employees from the IT department earning more than 70000.
Task 3	Update the salary of all HR employees by 10% increase.

Task 4	Delete all employees who joined before 2020 (use LIMIT 100).
Task 5	Write a SELECT query with pagination: show page 2 of results, 5 rows per page.

Chapter 5 — Built-in Functions

MySQL provides hundreds of built-in functions for string manipulation, math, date handling, and type conversion. These are essential for data transformation in queries.

String Functions

SELECT UPPER('hello');	-- HELLO
SELECT LOWER('WORLD');	-- world
SELECT LENGTH('MySQL');	-- 5
SELECT CHAR_LENGTH('MySQL');	-- 5 (for multibyte too)
SELECT CONCAT('Hello', ' ', 'World');	-- Hello World
SELECT CONCAT_WS(',', ' ', 'Alice', 'Bob');	-- Alice, Bob
SELECT SUBSTRING('MySQL', 2, 3);	-- ySQL -> MySQL... wait: ySQL
SELECT LEFT('MySQL', 2);	-- My
SELECT RIGHT('MySQL', 3);	-- SQL
SELECT TRIM(' hello ');	-- hello
SELECT LTRIM(' hello');	-- hello
SELECT RTRIM('hello ');	-- hello
SELECT REPLACE('Hello World', 'World', 'MySQL');	-- Hello MySQL
SELECT INSTR('Hello', 'ell');	-- 2
SELECT LPAD('5', 3, '0');	-- 005
SELECT RPAD('A', 4, '-');	-- A---
SELECT REVERSE('MySQL');	-- LQSyM
SELECT FORMAT(1234567.89, 2);	-- 1,234,567.89

Numeric Functions

SELECT ABS(-15);	-- 15
SELECT ROUND(3.7456, 2);	-- 3.75
SELECT CEIL(4.1);	-- 5
SELECT FLOOR(4.9);	-- 4
SELECT MOD(10, 3);	-- 1
SELECT POWER(2, 8);	-- 256
SELECT SQRT(144);	-- 12
SELECT TRUNCATE(3.999, 1);	-- 3.9
SELECT RAND();	-- random 0-1
SELECT FLOOR(RAND()*100);	-- random 0-99

Date & Time Functions

SELECT NOW();	-- 2024-01-15 10:30:00
SELECT CURDATE();	-- 2024-01-15
SELECT CURTIME();	-- 10:30:00
SELECT DATE(NOW());	-- 2024-01-15
SELECT YEAR(NOW()), MONTH(NOW()), DAY(NOW());	
SELECT DAYNAME(NOW());	-- Monday
SELECT MONTHNAME(NOW());	-- January
SELECT DAYOFWEEK(NOW());	-- 1=Sunday
SELECT DATEDIFF('2024-12-31', '2024-01-01');	-- days between

```
SELECT DATE_ADD(NOW(), INTERVAL 30 DAY);
SELECT DATE_SUB(NOW(), INTERVAL 1 YEAR);
SELECT DATE_FORMAT(NOW(), '%d/%m/%Y %H:%i'); -- 15/01/2024 10:30
SELECT TIMESTAMPDIFF(YEAR, '1995-01-01', NOW()); -- age
SELECT UNIX_TIMESTAMP(); -- epoch seconds
SELECT FROM_UNIXTIME(1700000000); -- human readable
```

Aggregate Functions

```
SELECT COUNT(*) FROM employees;
SELECT COUNT(DISTINCT dept) FROM employees;
SELECT SUM(salary) FROM employees;
SELECT AVG(salary) FROM employees;
SELECT MAX(salary) FROM employees;
SELECT MIN(salary) FROM employees;
SELECT GROUP_CONCAT(name ORDER BY name SEPARATOR ', ') FROM employees;
```

Conditional / Control Functions

```
-- IF
SELECT name, IF(salary > 70000, 'Senior', 'Junior') AS level FROM employees;

-- IFNULL / COALESCE
SELECT IFNULL(phone, 'N/A') FROM employees;
SELECT COALESCE(phone, email, 'No contact') FROM employees;

-- NULLIF
SELECT NULLIF(dept, 'Unknown') FROM employees; -- returns NULL if match

-- CASE
SELECT name,
CASE
    WHEN salary >= 90000 THEN 'A'
    WHEN salary >= 70000 THEN 'B'
    ELSE 'C'
END AS grade
FROM employees;
```

 Practice Tasks	
Task 1	Write a query to show each employee's name in uppercase and their email domain extracted using SUBSTRING_INDEX.
Task 2	Calculate the number of years each employee has worked using TIMESTAMPDIFF.
Task 3	Format salaries with 2 decimal places and thousand separators using FORMAT.
Task 4	Use CASE to label employees as 'High', 'Mid', 'Low' based on salary range.
Task 5	Use GROUP_CONCAT to show all employees in each department as a comma-separated list.

Chapter 6 — GROUP BY / HAVING

GROUP BY aggregates rows sharing the same values into summary rows. HAVING filters those groups — it's the WHERE clause for aggregated data. These are essential for analytics and reports.

GROUP BY

```
-- Count employees per department
SELECT dept, COUNT(*) AS total
FROM employees
GROUP BY dept;

-- Average salary per department
SELECT dept, AVG(salary) AS avg_salary, MAX(salary) AS max_salary
FROM employees
GROUP BY dept;

-- Group by multiple columns
SELECT dept, YEAR(joined) AS year, COUNT(*) AS hired
FROM employees
GROUP BY dept, YEAR(joined)
ORDER BY dept, year;
```

HAVING

```
-- Departments with more than 3 employees
SELECT dept, COUNT(*) AS total
FROM employees
GROUP BY dept
HAVING COUNT(*) > 3;

-- Departments where avg salary > 70000
SELECT dept, AVG(salary) AS avg_sal
FROM employees
GROUP BY dept
HAVING avg_sal > 70000;

-- Departments with min salary > 50000
SELECT dept, MIN(salary) AS min_sal
FROM employees
GROUP BY dept
HAVING min_sal > 50000
ORDER BY min_sal DESC;
```

WHERE vs HAVING

```
-- WHERE filters BEFORE grouping (filters rows)
-- HAVING filters AFTER grouping (filters groups)

SELECT dept, COUNT(*) AS total
```

FROM employees
WHERE salary > 50000 -- filter rows first
GROUP BY dept
HAVING COUNT(*) >= 2 -- then filter groups
ORDER BY total DESC
LIMIT 5;

WITH ROLLUP

-- Adds subtotals and grand total
SELECT dept, SUM(salary) AS total_salary
FROM employees
GROUP BY dept WITH ROLLUP;
-- NULL in dept column = grand total row

 INFO

ORDER BY and LIMIT in GROUP BY: apply after HAVING. Full query order: WHERE → GROUP BY → HAVING → ORDER BY → LIMIT.

🔄 Practice Tasks	
Task 1	Find total salary expenditure per department.
Task 2	Find departments where average salary exceeds 65000.
Task 3	Find the year with most employee hires across all departments.
Task 4	List departments that have at least 2 employees earning above 70000.
Task 5	Use WITH ROLLUP to show department totals and a grand total.

Chapter 7 — Joins

JOINS combine rows from two or more tables based on related columns. They are the most powerful feature of relational databases and are used in almost every real-world query.

Setup for Examples

```
-- customers table: id, name, city
-- orders table: order_id, customer_id, amount, order_date
-- products table: product_id, name, price
-- order_items table: order_id, product_id, qty
```

INNER JOIN — Only Matching Rows

```
SELECT c.name, o.order_id, o.amount
FROM customers c
INNER JOIN orders o ON c.id = o.customer_id;
-- Returns only customers who HAVE orders
```

LEFT JOIN — All Left + Matching Right

```
SELECT c.name, o.order_id, o.amount
FROM customers c
LEFT JOIN orders o ON c.id = o.customer_id;
-- Returns ALL customers, NULL for order columns if no orders

-- Find customers with NO orders
SELECT c.name FROM customers c
LEFT JOIN orders o ON c.id = o.customer_id
WHERE o.order_id IS NULL;
```

RIGHT JOIN — All Right + Matching Left

```
SELECT c.name, o.order_id
FROM customers c
RIGHT JOIN orders o ON c.id = o.customer_id;
-- Returns ALL orders, NULL for customer if unmatched
```

CROSS JOIN — Cartesian Product

```
SELECT c.name, p.name AS product
FROM customers c
CROSS JOIN products p;
-- Every customer × every product (use carefully!)
```

SELF JOIN — Table Joins Itself

```
-- employees table with manager_id referencing emp_id
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id;
```

Multiple Joins

```
SELECT c.name, o.order_id, p.name AS product, oi.qty
FROM customers c
JOIN orders o ON c.id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
WHERE c.city = 'Mumbai'
ORDER BY o.order_date DESC;
```

USING Clause (shorthand)

```
-- When column names are the same in both tables
SELECT c.name, o.amount
FROM customers c
JOIN orders o USING (customer_id);
```

 Practice Tasks	
Task 1	List all customers and their total order amount (include customers with no orders).
Task 2	Find all products that have never been ordered (use LEFT JOIN + IS NULL).
Task 3	Show each employee along with their manager's name using a SELF JOIN.
Task 4	Get a list of orders showing customer name, product name, qty, and order date.
Task 5	Count orders per customer — include customers with 0 orders.

Chapter 8 — Subqueries

A subquery is a query nested inside another query. Subqueries can appear in SELECT, FROM, WHERE, and HAVING clauses. They are powerful for filtering, transforming, and comparing data.

Subquery in WHERE

```
-- Employees earning above average salary
SELECT name, salary FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);

-- Employees in departments with most staff
SELECT name FROM employees
WHERE dept = (
    SELECT dept FROM employees
    GROUP BY dept ORDER BY COUNT(*) DESC LIMIT 1
);
```

Subquery with IN / NOT IN

```
-- Customers who placed orders
SELECT name FROM customers
WHERE id IN (SELECT DISTINCT customer_id FROM orders);

-- Customers with NO orders
SELECT name FROM customers
WHERE id NOT IN (SELECT customer_id FROM orders WHERE customer_id IS NOT NULL);
```

Correlated Subquery

```
-- Each employee's salary vs their dept average
SELECT name, dept, salary,
    (SELECT AVG(salary) FROM employees e2
     WHERE e2.dept = e1.dept) AS dept_avg
FROM employees e1;

-- Employees who earn more than their dept average
SELECT name, dept, salary FROM employees e1
WHERE salary > (
    SELECT AVG(salary) FROM employees e2
    WHERE e2.dept = e1.dept
);
```

Subquery in FROM (Derived Table)

```
SELECT dept, avg_sal
FROM (
    SELECT dept, AVG(salary) AS avg_sal
    FROM employees
    GROUP BY dept
)
```

```
) AS dept_summary
WHERE avg_sal > 70000;
```

Subquery in SELECT

```
SELECT name, salary,
       (SELECT COUNT(*) FROM orders o WHERE o.emp_id = e.emp_id) AS order_count
FROM employees e;
```

EXISTS / NOT EXISTS

```
-- Customers who have at least one order
SELECT name FROM customers c
WHERE EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.id
);
```

```
-- Faster than IN for large datasets with NULLs
```



TIP

Use EXISTS instead of IN when the subquery result could contain NULLs — IN with NULLs can give unexpected results.

Practice Tasks

Task 1	Find employees who earn more than the overall average salary.
Task 2	Find products that have never been included in any order using NOT IN.
Task 3	For each employee, show their salary and the maximum salary in their department using a correlated subquery.
Task 4	Use a derived table to find which departments have above-average headcount.
Task 5	Use EXISTS to find customers who placed more than 3 orders.

Chapter 9 — Views

A VIEW is a virtual table based on a SELECT query. It doesn't store data itself but provides a reusable, named query. Views simplify complex queries, improve security, and provide abstraction.

Create a View

```
CREATE VIEW it_employees AS
SELECT emp_id, name, salary FROM employees
WHERE dept = 'IT';

-- View with joins
CREATE VIEW order_details AS
SELECT o.order_id, c.name AS customer, o.amount, o.order_date
FROM orders o
JOIN customers c ON o.customer_id = c.id;
```

Use a View

```
SELECT * FROM it_employees;
SELECT * FROM it_employees WHERE salary > 80000;
SELECT * FROM order_details ORDER BY order_date DESC;
```

Show Views

```
SHOW FULL TABLES WHERE TABLE_TYPE = 'VIEW';
SHOW CREATE VIEW it_employees;
```

Update / Replace a View

```
CREATE OR REPLACE VIEW it_employees AS
SELECT emp_id, name, salary, joined FROM employees
WHERE dept = 'IT';

ALTER VIEW it_employees AS
SELECT emp_id, name FROM employees WHERE dept = 'IT';
```

Drop a View

```
DROP VIEW it_employees;
DROP VIEW IF EXISTS it_employees;
```

Updatable Views

```
-- Views without GROUP BY, DISTINCT, subqueries are updatable
UPDATE it_employees SET salary = 90000 WHERE emp_id = 1;

-- Make read-only with WITH CHECK OPTION
CREATE VIEW it_employees AS
SELECT * FROM employees WHERE dept = 'IT'
WITH CHECK OPTION;
```

```
-- Now INSERT/UPDATE must satisfy WHERE dept = 'IT'
```

WITH CHECK OPTION

```
-- Prevents inserting/updating rows that would 'escape' the view
```

```
CREATE VIEW high_earners AS
```

```
SELECT * FROM employees WHERE salary > 80000
```

```
WITH CHECK OPTION;
```

```
-- This will FAIL:
```

```
INSERT INTO high_earners (name, salary) VALUES ('Test', 50000);
```

INFO

Views with GROUP BY, DISTINCT, aggregate functions, UNION, or subqueries in WHERE are NOT updatable.

Practice Tasks

Task 1	Create a view showing only active employees (status = 'active').
Task 2	Create a view joining orders and customers showing total order value per customer.
Task 3	Query your view filtering to orders greater than 5000.
Task 4	Add a WITH CHECK OPTION to prevent inserting inactive employees via the view.
Task 5	Drop and recreate a view using CREATE OR REPLACE.

Chapter 10 — Indexes

Indexes speed up data retrieval by creating a separate data structure (B-Tree, Hash, or Full-Text). Without indexes, MySQL does a full table scan for every query. Proper indexing is the #1 performance optimization.

Types of Indexes

PRIMARY KEY	-- Clustered index, every table should have one
UNIQUE	-- No duplicates, used for columns like email
INDEX / KEY	-- Regular index for faster lookups
FULLTEXT	-- Full-text search on TEXT/VARCHAR columns
COMPOSITE	-- Index on multiple columns
SPATIAL	-- Geographic data (rarely used)

Create Index

```
-- At table creation
CREATE TABLE employees (
  emp_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  email VARCHAR(200) UNIQUE,
  dept VARCHAR(50),
  INDEX idx_dept (dept) -- Regular index
);

-- After creation
CREATE INDEX idx_name ON employees (name);
CREATE UNIQUE INDEX idx_email ON employees (email);
CREATE INDEX idx_dept_salary ON employees (dept, salary); -- Composite
CREATE FULLTEXT INDEX idx_ft_name ON employees (name);

-- Using ALTER TABLE
ALTER TABLE employees ADD INDEX idx_joined (joined);
```

Show / Drop Indexes

```
SHOW INDEX FROM employees;
SHOW INDEXES FROM employees\G

DROP INDEX idx_name ON employees;
ALTER TABLE employees DROP INDEX idx_name;
```

Using EXPLAIN to Verify Index Usage

```
EXPLAIN SELECT * FROM employees WHERE dept = 'IT';
-- type: ALL = full scan (bad), ref/range = index used (good)

EXPLAIN FORMAT=JSON SELECT * FROM employees WHERE dept = 'IT'\G
```

Full-Text Search

```
-- Natural language mode (default)
SELECT * FROM articles
WHERE MATCH(title, body) AGAINST ('MySQL performance');

-- Boolean mode
SELECT * FROM articles
WHERE MATCH(title) AGAINST ('+MySQL -Oracle' IN BOOLEAN MODE);

-- Query expansion
SELECT * FROM articles
WHERE MATCH(title) AGAINST ('database' WITH QUERY EXPANSION);
```

Index Best Practices

```
-- Index columns used in WHERE, JOIN ON, ORDER BY
-- Avoid indexing low-cardinality columns (gender, boolean)
-- Composite index: most selective column first
-- Covering index: all columns in SELECT are in index
CREATE INDEX idx_cover ON employees (dept, salary, name);
-- This covers: SELECT name, salary FROM employees WHERE dept = 'IT'
-- No table access needed!
```

 NOTE

Too many indexes slow down INSERT/UPDATE/DELETE since indexes must be updated. Index strategically, not blindly.

 Practice Tasks	
Task 1	Create an index on the email column of users table and verify with SHOW INDEX.
Task 2	Create a composite index on (dept, salary) and test with EXPLAIN.
Task 3	Create a FULLTEXT index on a product_description column and search it.
Task 4	Use EXPLAIN to compare query performance with and without an index.
Task 5	Drop an index and observe the EXPLAIN output change.

Chapter 11 — Authorization & Privileges

MySQL uses a privilege system to control who can do what. Privileges are granted at the global, database, table, or column level. This is critical for security in multi-user environments.

Create a User

```
CREATE USER 'alice'@'localhost' IDENTIFIED BY 'StrongPass@123';
CREATE USER 'bob'@'%' IDENTIFIED BY 'AnotherPass@456'; -- Any host
CREATE USER 'readonly'@'192.168.1.%' IDENTIFIED BY 'ReadOnly@789';
```

Grant Privileges

```
-- All privileges on all databases (superuser)
GRANT ALL PRIVILEGES ON *.* TO 'alice'@'localhost';

-- All privileges on one database
GRANT ALL PRIVILEGES ON company.* TO 'bob'@'%;

-- Specific privileges on a table
GRANT SELECT, INSERT ON company.employees TO 'readonly'@'localhost';

-- Column-level privilege
GRANT SELECT (name, dept) ON company.employees TO 'viewer'@'localhost';

-- Grant with ability to pass to others
GRANT SELECT ON company.* TO 'manager'@'%' WITH GRANT OPTION;

FLUSH PRIVILEGES; -- Apply changes (not needed for GRANT/REVOKE but safe habit)
```

Revoke Privileges

```
REVOKE INSERT ON company.employees FROM 'bob'@'%;
REVOKE ALL PRIVILEGES ON company.* FROM 'bob'@'%;
REVOKE GRANT OPTION ON *.* FROM 'manager'@'%;
```

Show Privileges

```
SHOW GRANTS; -- Current user
SHOW GRANTS FOR 'alice'@'localhost';
SHOW GRANTS FOR CURRENT_USER();
```

User Management

```
-- Change password
ALTER USER 'alice'@'localhost' IDENTIFIED BY 'NewPass@999';

-- Rename user
RENAME USER 'alice'@'localhost' TO 'alicia'@'localhost';
```

```
-- Lock/unlock user
ALTER USER 'alice'@'localhost' ACCOUNT LOCK;
ALTER USER 'alice'@'localhost' ACCOUNT UNLOCK;

-- Drop user
DROP USER 'alice'@'localhost';
DROP USER IF EXISTS 'alice'@'localhost';

-- List all users
SELECT user, host FROM mysql.user;
```

Privilege Types Reference

SELECT	-- Read data
INSERT	-- Add rows
UPDATE	-- Modify rows
DELETE	-- Remove rows
CREATE	-- Create databases/tables
DROP	-- Delete databases/tables
ALTER	-- Modify table structure
INDEX	-- Create/drop indexes
EXECUTE	-- Run stored procedures
SUPER	-- Admin operations
GRANT OPTION	-- Pass privileges to others

 Practice Tasks	
Task 1	Create a user 'reporter' who can only SELECT on the company database.
Task 2	Create a user 'developer' with SELECT, INSERT, UPDATE, DELETE on company.*
Task 3	Show all privileges for 'reporter' user.
Task 4	Revoke INSERT from 'developer' on the employees table only.
Task 5	Lock the 'reporter' account and then unlock it.

Chapter 12 — Roles

Roles (MySQL 8.0+) are named collections of privileges. Instead of granting the same set of privileges to many users individually, you create a role, grant it privileges, then assign the role to users.

Create and Grant to Roles

```
-- Create roles
CREATE ROLE 'read_only', 'read_write', 'admin_role';

-- Grant privileges to roles
GRANT SELECT ON company.* TO 'read_only';
GRANT SELECT, INSERT, UPDATE, DELETE ON company.* TO 'read_write';
GRANT ALL PRIVILEGES ON company.* TO 'admin_role';
```

Assign Roles to Users

```
-- Assign role to user
GRANT 'read_only' TO 'reporter'@'localhost';
GRANT 'read_write' TO 'developer'@'localhost';
GRANT 'admin_role' TO 'admin'@'localhost';

-- Assign multiple roles
GRANT 'read_only', 'read_write' TO 'developer'@'localhost';
```

Set Default Role & Activate

```
-- Set default role (auto-activated on login)
SET DEFAULT ROLE 'read_only' TO 'reporter'@'localhost';
SET DEFAULT ROLE ALL TO 'developer'@'localhost';

-- Activate role in session (if not default)
SET ROLE 'read_write';
SET ROLE ALL;
SET ROLE NONE; -- Deactivate all roles

-- Check current active roles
SELECT CURRENT_ROLE();
```

Show & Manage Roles

```
SHOW GRANTS FOR 'reporter'@'localhost';
SHOW GRANTS FOR 'read_only';

-- Revoke role from user
REVOKE 'read_only' FROM 'reporter'@'localhost';

-- Drop a role
DROP ROLE 'read_only';
DROP ROLE IF EXISTS 'read_only';
```

```
-- List all roles
```

```
SELECT * FROM mysql.user WHERE account_locked='Y' AND password_expired='Y';
```

 TIP

Roles simplify privilege management enormously in multi-user systems. Define roles by job function, not by individual.

Practice Tasks

Task 1	Create three roles: viewer, editor, manager with appropriate privileges on company database.
Task 2	Create users for each role and assign them the roles.
Task 3	Set viewer role as default for the viewer user.
Task 4	Activate the editor role in a session and verify with <code>CURRENT_ROLE()</code> .
Task 5	Revoke the manager role from a user and drop the role.

Chapter 13 — Transactions

A transaction is a sequence of SQL statements treated as a single unit. Either all succeed (COMMIT) or all are undone (ROLLBACK). Transactions ensure ACID properties: Atomicity, Consistency, Isolation, Durability.

Basic Transaction

```
START TRANSACTION;  -- or BEGIN;

UPDATE accounts SET balance = balance - 1000 WHERE id = 1;
UPDATE accounts SET balance = balance + 1000 WHERE id = 2;

COMMIT;  -- Save changes permanently
-- or
ROLLBACK; -- Undo all changes since START TRANSACTION
```

SAVEPOINT — Partial Rollback

```
START TRANSACTION;

INSERT INTO orders (customer_id, amount) VALUES (1, 500);
SAVEPOINT after_insert;

INSERT INTO order_items (order_id, product_id, qty) VALUES (1, 5, 2);

-- Something went wrong with items, rollback to savepoint
ROLLBACK TO SAVEPOINT after_insert;

-- Order is still intact, commit it
COMMIT;

-- Release savepoint (optional)
RELEASE SAVEPOINT after_insert;
```

Auto-Commit

```
-- Check current setting
SELECT @@autocommit;

-- Disable auto-commit
SET autocommit = 0;

-- Now every query needs explicit COMMIT
INSERT INTO employees (name) VALUES ('Test');
COMMIT;

-- Re-enable
SET autocommit = 1;
```

Transaction Isolation Levels

```
-- Set isolation level
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
SET GLOBAL TRANSACTION ISOLATION LEVEL REPEATABLE READ;

-- Check current level
SELECT @@transaction_isolation;

-- Levels (from least to most strict):
-- READ UNCOMMITTED -- Dirty reads possible
-- READ COMMITTED -- No dirty reads
-- REPEATABLE READ -- Default in MySQL/InnoDB
-- SERIALIZABLE -- Strictest, full locking
```

Error Handling Pattern

```
START TRANSACTION;

UPDATE accounts SET balance = balance - 5000 WHERE id = 1;

-- Check if sufficient balance
SELECT balance INTO @bal FROM accounts WHERE id = 1;

IF @bal < 0 THEN
    ROLLBACK;
    SELECT 'Insufficient funds' AS error;
ELSE
    UPDATE accounts SET balance = balance + 5000 WHERE id = 2;
    COMMIT;
    SELECT 'Transfer successful' AS result;
END IF;
```

NOTE

InnoDB (default MySQL engine) supports transactions. MyISAM does NOT support transactions.

Practice Tasks

Task 1	Write a transaction to transfer 500 from account A to account B.
Task 2	Use SAVEPOINT to partially roll back an order insertion without losing customer data.
Task 3	Disable autocommit, run 3 inserts, then commit or rollback based on a condition.
Task 4	Check and change the transaction isolation level to SERIALIZABLE for a session.
Task 5	Simulate a failed transaction (force an error) and verify ROLLBACK worked.

Chapter 14 — Stored Procedures

A Stored Procedure is a precompiled block of SQL code stored in the database. It can accept parameters, contain logic (IF/LOOP/WHILE), and perform multiple operations. Reduces network round-trips and centralizes business logic.

Basic Stored Procedure

```
DELIMITER //
```

```
CREATE PROCEDURE GetAllEmployees()  
BEGIN  
    SELECT * FROM employees;  
END //
```

```
DELIMITER ;
```

```
-- Call it  
CALL GetAllEmployees();
```

Procedure with Parameters

```
DELIMITER //
```

```
CREATE PROCEDURE GetEmployeesByDept(IN dept_name VARCHAR(50))  
BEGIN  
    SELECT * FROM employees WHERE dept = dept_name;  
END //
```

```
DELIMITER ;
```

```
CALL GetEmployeesByDept('IT');
```

IN, OUT, INOUT Parameters

```
DELIMITER //
```

```
CREATE PROCEDURE GetEmpCount(  
    IN dept_name VARCHAR(50),  
    OUT emp_count INT  
)  
BEGIN  
    SELECT COUNT(*) INTO emp_count  
    FROM employees WHERE dept = dept_name;  
END //
```

```
DELIMITER ;
```

```
CALL GetEmpCount('IT', @count);
```

```
SELECT @count;
```

Variables and Logic

```
DELIMITER //

CREATE PROCEDURE GiveSalaryHike(IN dept_name VARCHAR(50), IN pct DECIMAL(5,2))
BEGIN
    DECLARE affected INT DEFAULT 0;

    UPDATE employees SET salary = salary * (1 + pct/100)
    WHERE dept = dept_name;

    SET affected = ROW_COUNT();
    SELECT CONCAT(affected, ' employees updated') AS result;
END //

DELIMITER ;

CALL GiveSalaryHike('IT', 10);
```

IF / ELSEIF / ELSE

```
DELIMITER //

CREATE PROCEDURE CheckSalaryGrade(IN emp_id INT)
BEGIN
    DECLARE sal DECIMAL(10,2);
    SELECT salary INTO sal FROM employees WHERE id = emp_id;
    IF sal >= 90000 THEN
        SELECT 'Grade A' AS grade;
    ELSEIF sal >= 70000 THEN
        SELECT 'Grade B' AS grade;
    ELSE
        SELECT 'Grade C' AS grade;
    END IF;
END //

DELIMITER ;
```

Loops in Procedures

```
-- WHILE loop
DELIMITER //
CREATE PROCEDURE CountTo(IN n INT)
BEGIN
    DECLARE i INT DEFAULT 1;
    WHILE i <= n DO
        INSERT INTO count_log (num) VALUES (i);
        SET i = i + 1;
    END WHILE;
END //
DELIMITER ;

-- LOOP with LEAVE
DELIMITER //
```

```
CREATE PROCEDURE LoopExample()  
BEGIN  
  DECLARE i INT DEFAULT 1;  
  myloop: LOOP  
    IF i > 5 THEN LEAVE myloop; END IF;  
    SET i = i + 1;  
  END LOOP myloop;  
END //  
DELIMITER ;
```

Show / Drop Procedures

```
SHOW PROCEDURE STATUS WHERE Db = 'company';  
SHOW CREATE PROCEDURE GetAllEmployees;  
DROP PROCEDURE IF EXISTS GetAllEmployees;
```

 Practice Tasks	
Task 1	Create a procedure that accepts an employee ID and returns their full details.
Task 2	Create a procedure with OUT parameter that returns total salary of a department.
Task 3	Create a procedure to insert a new employee and return the new auto-increment ID.
Task 4	Create a procedure with a WHILE loop to log salary history for all employees.
Task 5	Drop and recreate a procedure with improved error handling.

Chapter 15 — User-Defined Functions

User-Defined Functions (UDFs) return a single value and can be used directly in SQL queries, just like built-in functions. Unlike procedures, functions cannot use OUT parameters or call COMMIT/ROLLBACK.

Create a Function

```
DELIMITER //
```

```
CREATE FUNCTION GetFullName(first VARCHAR(50), last VARCHAR(50))  
RETURNS VARCHAR(100)  
DETERMINISTIC  
BEGIN  
    RETURN CONCAT(first, ' ', last);  
END //
```

```
DELIMITER ;
```

```
-- Use in query  
SELECT GetFullName('John', 'Doe');  
SELECT GetFullName(first_name, last_name) FROM users;
```

Function with Logic

```
DELIMITER //
```

```
CREATE FUNCTION SalaryGrade(salary DECIMAL(10,2))  
RETURNS VARCHAR(10)  
DETERMINISTIC  
BEGIN  
    DECLARE grade VARCHAR(10);  
    IF salary >= 90000 THEN SET grade = 'A';  
    ELSEIF salary >= 70000 THEN SET grade = 'B';  
    ELSEIF salary >= 50000 THEN SET grade = 'C';  
    ELSE SET grade = 'D';  
    END IF;  
    RETURN grade;  
END //
```

```
DELIMITER ;
```

```
SELECT name, salary, SalaryGrade(salary) AS grade FROM employees;
```

Function Reading from Table

```
DELIMITER //
```

```
CREATE FUNCTION GetDeptAvgSalary(dept_name VARCHAR(50))
```

```

RETURNS DECIMAL(10,2)
READS SQL DATA
BEGIN
    DECLARE avg_sal DECIMAL(10,2);
    SELECT AVG(salary) INTO avg_sal
    FROM employees WHERE dept = dept_name;
    RETURN IFNULL(avg_sal, 0);
END //

DELIMITER ;

SELECT dept, GetDeptAvgSalary(dept) AS avg_salary FROM employees GROUP BY dept;

```

Function Characteristics

DETERMINISTIC	-- Same input always gives same output
NOT DETERMINISTIC	-- Output may vary (e.g., uses NOW())
READS SQL DATA	-- Reads from tables
MODIFIES SQL DATA	-- Inserts/updates (rare for functions)
NO SQL	-- No SQL statements inside

Show / Drop Functions

```

SHOW FUNCTION STATUS WHERE Db = 'company';
SHOW CREATE FUNCTION SalaryGrade;
DROP FUNCTION IF EXISTS SalaryGrade;

```

⚠ NOTE

If you get 'ERROR 1418: Binary logging is on', run: SET GLOBAL log_bin_trust_function_creators = 1;

🔗 Practice Tasks

Task 1	Create a function that takes a date and returns how many years ago it was.
Task 2	Create a function GetTaxAmount(salary) that returns 20% tax for >80000, else 10%.
Task 3	Use your function in a SELECT to show each employee's tax amount.
Task 4	Create a DETERMINISTIC function that formats a phone number to (xxx) xxx-xxxx.
Task 5	Create a function that reads from the employees table to get employee count by dept.

Chapter 16 — Triggers

A trigger is a stored program that automatically executes (fires) in response to a DML event (INSERT, UPDATE, DELETE) on a table. Triggers are used for auditing, validation, and automatic data maintenance.

Trigger Timing & Events

Timing:	BEFORE		AFTER		
Events:	INSERT		UPDATE		DELETE

-- 6 possible triggers per table:

BEFORE	INSERT		AFTER	INSERT
BEFORE	UPDATE		AFTER	UPDATE
BEFORE	DELETE		AFTER	DELETE

NEW and OLD Row References

NEW.column	-- New value (available in INSERT and UPDATE)
OLD.column	-- Old value (available in UPDATE and DELETE)

Trigger		NEW		OLD
BEFORE/AFTER INSERT		YES		NO
BEFORE/AFTER UPDATE		YES		YES
BEFORE/AFTER DELETE		NO		YES

Audit Log Trigger

```
-- Create audit table first
CREATE TABLE salary_audit (
  id INT AUTO_INCREMENT PRIMARY KEY,
  emp_id INT, old_salary DECIMAL(10,2),
  new_salary DECIMAL(10,2), changed_at DATETIME,
  changed_by VARCHAR(50)
);

DELIMITER //
CREATE TRIGGER trg_salary_update
AFTER UPDATE ON employees
FOR EACH ROW
BEGIN
  IF OLD.salary <> NEW.salary THEN
    INSERT INTO salary_audit
      VALUES (NULL, NEW.emp_id, OLD.salary, NEW.salary, NOW(), USER());
  END IF;
END //
DELIMITER ;
```

BEFORE Trigger — Validation

```

DELIMITER //
CREATE TRIGGER trg_validate_salary
BEFORE INSERT ON employees
FOR EACH ROW
BEGIN
    IF NEW.salary < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary cannot be negative';
    END IF;
END //
DELIMITER ;

```

Soft Delete Trigger

```

DELIMITER //
CREATE TRIGGER trg_soft_delete
BEFORE DELETE ON employees
FOR EACH ROW
BEGIN
    INSERT INTO deleted_employees SELECT *, NOW() FROM employees WHERE emp_id =
    OLD.emp_id;
END //
DELIMITER ;

```

Show / Drop Triggers

```

SHOW TRIGGERS FROM company;
SHOW TRIGGERS LIKE 'employees'\G
SHOW CREATE TRIGGER trg_salary_update;

DROP TRIGGER IF EXISTS trg_salary_update;

```

⚠ NOTE

Triggers fire FOR EACH ROW — if you update 1000 rows, trigger fires 1000 times. Keep trigger logic simple and fast.

🔗 Practice Tasks

Task 1	Create an AFTER INSERT trigger that logs new employee additions to an emp_log table.
Task 2	Create a BEFORE UPDATE trigger that prevents salary from being reduced.
Task 3	Create an AFTER DELETE trigger that archives deleted orders to orders_archive.
Task 4	Use SIGNAL in a BEFORE INSERT trigger to reject orders with qty <= 0.
Task 5	Show all triggers on the employees table.

Chapter 17 — Events

MySQL Events are scheduled tasks (like cron jobs) that run automatically at a specified time or on a recurring schedule. They are managed by the MySQL Event Scheduler.

Enable Event Scheduler

```
-- Check status
SHOW VARIABLES LIKE 'event_scheduler';

-- Enable
SET GLOBAL event_scheduler = ON;

-- Enable at server startup (in my.cnf)
-- event_scheduler = ON
```

One-Time Event

```
CREATE EVENT evt_cleanup_once
ON SCHEDULE AT NOW() + INTERVAL 1 HOUR
DO
    DELETE FROM temp_logs WHERE created_at < NOW() - INTERVAL 7 DAY;
```

Recurring Event

```
-- Every day at midnight
CREATE EVENT evt_daily_cleanup
ON SCHEDULE EVERY 1 DAY
STARTS '2024-01-01 00:00:00'
DO
    DELETE FROM sessions WHERE last_active < NOW() - INTERVAL 30 DAY;

-- Every hour
CREATE EVENT evt_hourly_stats
ON SCHEDULE EVERY 1 HOUR
DO
BEGIN
    INSERT INTO stats (metric, value, recorded_at)
    SELECT 'active_users', COUNT(*), NOW()
    FROM sessions WHERE last_active > NOW() - INTERVAL 1 HOUR;
END;

-- With end date
CREATE EVENT evt_promo
ON SCHEDULE EVERY 1 DAY
STARTS '2024-01-01' ENDS '2024-01-31'
DO INSERT INTO promo_log VALUES (NOW());
```

Manage Events

```

SHOW EVENTS FROM company;
SHOW CREATE EVENT evt_daily_cleanup;

-- Enable / Disable
ALTER EVENT evt_daily_cleanup ENABLE;
ALTER EVENT evt_daily_cleanup DISABLE;

-- Rename
ALTER EVENT evt_daily_cleanup RENAME TO evt_session_cleanup;

-- Drop
DROP EVENT IF EXISTS evt_daily_cleanup;

```

INFO

Events require event_scheduler = ON globally. They don't run on replicas by default, which is usually desired.

Practice Tasks

Task 1	Enable the event scheduler and verify it's running.
Task 2	Create a one-time event to archive old records 2 hours from now.
Task 3	Create a recurring event that runs every midnight to summarize daily sales.
Task 4	Disable a scheduled event without dropping it.
Task 5	List all events in your database and show details of one.

Chapter 18 — CTE (Common Table Expressions)

A CTE (Common Table Expression) is a named temporary result set defined using the WITH clause. CTEs make complex queries more readable, allow recursion, and can be referenced multiple times in one query.

Basic CTE

```
WITH high_earners AS (  
    SELECT * FROM employees WHERE salary > 80000  
)  
SELECT * FROM high_earners ORDER BY salary DESC;
```

Multiple CTEs

```
WITH  
dept_avg AS (  
    SELECT dept, AVG(salary) AS avg_sal FROM employees GROUP BY dept  
)  
above_avg AS (  
    SELECT e.name, e.dept, e.salary, d.avg_sal  
    FROM employees e  
    JOIN dept_avg d ON e.dept = d.dept  
    WHERE e.salary > d.avg_sal  
)  
SELECT * FROM above_avg ORDER BY dept, salary DESC;
```

Recursive CTE — Hierarchy

```
-- employees table: emp_id, name, manager_id  
WITH RECURSIVE emp_hierarchy AS (  
    -- Anchor: top-level (no manager)  
    SELECT emp_id, name, manager_id, 0 AS level  
    FROM employees WHERE manager_id IS NULL  
  
    UNION ALL  
  
    -- Recursive: join to children  
    SELECT e.emp_id, e.name, e.manager_id, h.level + 1  
    FROM employees e  
    JOIN emp_hierarchy h ON e.manager_id = h.emp_id  
)  
SELECT CONCAT(REPEAT(' ', level), name) AS hierarchy, level  
FROM emp_hierarchy  
ORDER BY level, name;
```

Recursive CTE — Number Series

```
WITH RECURSIVE numbers AS (  
    SELECT 1 AS n
```

```
UNION ALL
SELECT n + 1 FROM numbers WHERE n < 10
)
SELECT * FROM numbers;
-- Generates 1 through 10
```

CTE vs Subquery vs View

```
-- CTE: readable, reusable within same query, not stored
-- Subquery: inline, harder to read when complex
-- View: stored in DB, reusable across queries

-- CTEs cannot be indexed, but are great for readability
```

 Practice Tasks	
Task 1	Use a CTE to find the top 3 earners in each department.
Task 2	Write a query with 2 CTEs: one for dept totals, one for company average, then compare.
Task 3	Create a recursive CTE to list all employees in a management hierarchy.
Task 4	Use a CTE to paginate results without a subquery (CTE + LIMIT OFFSET).
Task 5	Refactor a complex nested subquery from Chapter 8 using a CTE.

Chapter 19 — Window Functions

Window functions perform calculations across a set of rows related to the current row (a 'window'), without collapsing rows like GROUP BY does. They are extremely powerful for analytics and ranking.

Syntax

```
function_name() OVER (  
  [PARTITION BY col1, col2] -- Group like GROUP BY but don't collapse  
  [ORDER BY col3]           -- Order within partition  
  [ROWS/RANGE frame]       -- Optional window frame  
)
```

Ranking Functions

```
SELECT name, dept, salary,  
  ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC) AS row_num,  
  RANK()         OVER (PARTITION BY dept ORDER BY salary DESC) AS rank_num,  
  DENSE_RANK()  OVER (PARTITION BY dept ORDER BY salary DESC) AS dense_rank,  
  NTILE(4)      OVER (ORDER BY salary DESC) AS quartile  
FROM employees;  
  
-- ROW_NUMBER: 1,2,3,4 (no ties)  
-- RANK:        1,2,2,4 (skips after tie)  
-- DENSE_RANK: 1,2,2,3 (no skips)
```

Top N Per Group

```
-- Top 2 earners per department  
WITH ranked AS (  
  SELECT *, RANK() OVER (PARTITION BY dept ORDER BY salary DESC) AS r  
  FROM employees  
)  
SELECT name, dept, salary FROM ranked WHERE r <= 2;
```

Aggregate Window Functions

```
SELECT name, dept, salary,  
  SUM(salary) OVER (PARTITION BY dept) AS dept_total,  
  AVG(salary) OVER (PARTITION BY dept) AS dept_avg,  
  COUNT(*)   OVER (PARTITION BY dept) AS dept_count,  
  MAX(salary) OVER () AS company_max  
FROM employees;
```

Running Totals — Frame Clause

```
SELECT order_date, amount,  
  SUM(amount) OVER (ORDER BY order_date  
  ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
  ) AS running_total  
FROM orders;
```

```
-- 3-month moving average
SELECT order_date, amount,
       AVG(amount) OVER (ORDER BY order_date
                        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
                        ) AS moving_avg_3
FROM orders;
```

LAG / LEAD — Access Adjacent Rows

```
SELECT order_date, amount,
       LAG(amount, 1) OVER (ORDER BY order_date) AS prev_amount,
       LEAD(amount, 1) OVER (ORDER BY order_date) AS next_amount,
       amount - LAG(amount,1) OVER (ORDER BY order_date) AS change
FROM orders;
```

FIRST_VALUE / LAST_VALUE / NTH_VALUE

```
SELECT name, dept, salary,
       FIRST_VALUE(name) OVER (PARTITION BY dept ORDER BY salary DESC) AS top_earner,
       NTH_VALUE(salary,2) OVER (PARTITION BY dept ORDER BY salary DESC) AS 2nd_salary
FROM employees;
```

Practice Tasks

Task 1	Rank all employees by salary within each department using DENSE_RANK.
Task 2	Find the top 3 earners per department using RANK in a CTE.
Task 3	Calculate running total of order amounts ordered by date.
Task 4	Calculate month-over-month revenue change using LAG.
Task 5	Show each employee's salary as a percentage of their department's total salary.

Chapter 20 — JSON in MySQL

MySQL 5.7+ supports native JSON data type with built-in functions to store, query, and manipulate JSON documents. Ideal for flexible/semi-structured data without schema changes.

JSON Column & Insert

```
CREATE TABLE products (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100),  
  specs JSON  
);  
  
INSERT INTO products (name, specs) VALUES  
(  
  'Laptop', '{"brand": "Dell", "ram": 16, "storage": "512GB", "tags":  
    ["fast", "portable"]}'  
,  
  'Phone', '{"brand": "Apple", "model": "15 Pro", "color": ["black", "white"]}'  
);
```

Reading JSON Data

```
-- Arrow operator (shorthand for JSON_EXTRACT)  
SELECT name, specs->'$.brand' AS brand FROM products;  
SELECT name, specs->'$.ram' AS ram FROM products;  
  
-- JSON_EXTRACT  
SELECT JSON_EXTRACT(specs, '$.brand') FROM products;  
  
-- Access array element  
SELECT specs->'$.tags[0]' FROM products WHERE id = 1;  
  
-- Nested  
SELECT specs->'$.dimensions.width' FROM products;
```

Modifying JSON

```
-- JSON_SET (update or insert)  
UPDATE products  
SET specs = JSON_SET(specs, '$.price', 999.99)  
WHERE id = 1;  
  
-- JSON_INSERT (only inserts if key doesn't exist)  
UPDATE products SET specs = JSON_INSERT(specs, '$.warranty', '2 years') WHERE id = 1;  
  
-- JSON_REPLACE (only updates existing)  
UPDATE products SET specs = JSON_REPLACE(specs, '$.ram', 32) WHERE id = 1;  
  
-- JSON_REMOVE  
UPDATE products SET specs = JSON_REMOVE(specs, '$.color') WHERE id = 2;
```

JSON Array Functions

```
SELECT JSON_ARRAY(1, 2, 3); -- [1, 2, 3]
SELECT JSON_ARRAY_APPEND(specs, '$.tags', 'new') FROM products;
SELECT JSON_LENGTH(specs->>'$.tags') FROM products;
SELECT JSON_CONTAINS(specs, '"fast"', '$.tags') FROM products;
```

JSON Object Functions

```
SELECT JSON_OBJECT('name', 'Alice', 'age', 30);
SELECT JSON_KEYS(specs) FROM products;
SELECT JSON_TYPE(specs) FROM products; -- OBJECT
SELECT JSON_VALID(specs) FROM products; -- 1 = valid
SELECT JSON_UNQUOTE(specs->>'$.brand') FROM products;
```

Search & Index JSON

```
-- Search in JSON
SELECT * FROM products WHERE specs->>'$.brand' = 'Dell';
SELECT * FROM products WHERE JSON_CONTAINS(specs, '16', '$.ram');

-- Virtual generated column for indexing
ALTER TABLE products
  ADD COLUMN brand VARCHAR(50) GENERATED ALWAYS AS (specs->>'$.brand') VIRTUAL;
CREATE INDEX idx_brand ON products (brand);
```

Practice Tasks

Task 1	Create a users table with a JSON preferences column; insert 3 users with different preferences.
Task 2	Query all users where preferences->theme = 'dark'.
Task 3	Update a specific JSON key for one user without replacing the whole document.
Task 4	Add a new key to all JSON records using JSON_SET.
Task 5	Create a virtual generated column from a JSON field and index it.

Chapter 21 — Query Optimization

Query optimization is about making queries run faster and consume fewer resources. MySQL's query optimizer tries to find the best execution plan, but you must understand EXPLAIN, indexes, and query patterns to guide it.

EXPLAIN

```
EXPLAIN SELECT * FROM employees WHERE dept = 'IT';
EXPLAIN FORMAT=JSON SELECT * FROM employees WHERE dept = 'IT'\G
EXPLAIN ANALYZE SELECT ... -- MySQL 8.0+ actually runs and reports

-- Key columns to check:
-- type: ALL(worst) > index > range > ref > eq_ref > const(best)
-- key: which index was used (NULL = no index)
-- rows: estimated rows scanned
-- Extra: Using filesort, Using temporary (bad), Using index (good)
```

Query Profiling

```
-- Enable profiling
SET profiling = 1;

-- Run your query
SELECT * FROM employees WHERE dept = 'IT';

-- Show profiles
SHOW PROFILES;
SHOW PROFILE FOR QUERY 1;
SHOW PROFILE CPU, MEMORY FOR QUERY 1;
```

Common Optimization Techniques

```
-- 1. Use indexes on WHERE, JOIN, ORDER BY columns
-- 2. Select only needed columns (avoid SELECT *)
-- 3. Use LIMIT to reduce result set
-- 4. Avoid functions on indexed columns in WHERE:
--   Bad:  WHERE YEAR(joined) = 2023
--   Good: WHERE joined BETWEEN '2023-01-01' AND '2023-12-31'

-- 5. Use covering indexes
CREATE INDEX idx_cover ON employees (dept, salary, name);
-- Now: SELECT name, salary FROM employees WHERE dept = 'IT'
-- ...doesn't need to read the actual table row!

-- 6. Avoid OR on different columns (use UNION instead)
--   Slow: WHERE dept = 'IT' OR name LIKE 'A%'
--   Faster:
SELECT * FROM employees WHERE dept = 'IT'
UNION
```

```

SELECT * FROM employees WHERE name LIKE 'A%';

-- 7. Use EXISTS instead of IN with NULL-possible subqueries
-- 8. Avoid DISTINCT if not needed
-- 9. Use STRAIGHT_JOIN to force join order (advanced)

```

Optimizer Hints (MySQL 8.0+)

```

-- Force an index
SELECT * FROM employees USE INDEX (idx_dept) WHERE dept = 'IT';
SELECT * FROM employees FORCE INDEX (idx_dept) WHERE dept = 'IT';
SELECT * FROM employees IGNORE INDEX (idx_dept) WHERE dept = 'IT';

-- Optimizer hints
SELECT /*+ NO_INDEX(employees idx_dept) */ * FROM employees WHERE dept = 'IT';

```

Slow Query Log

```

-- Enable slow query log
SET GLOBAL slow_query_log = 'ON';
SET GLOBAL long_query_time = 1; -- Log queries > 1 second
SET GLOBAL slow_query_log_file = '/var/log/mysql/slow.log';

-- Check settings
SHOW VARIABLES LIKE 'slow_query%';
SHOW VARIABLES LIKE 'long_query_time';

```

Practice Tasks

Task 1	Run EXPLAIN on a query with a WHERE clause — identify if an index is used.
Task 2	Create a covering index and verify EXPLAIN shows 'Using index'.
Task 3	Rewrite a slow WHERE clause with a function to use a range instead.
Task 4	Enable slow query log and run a query without an index to capture it.
Task 5	Compare EXPLAIN output of a query with and without USE INDEX hint.

Chapter 22 — Partitioning

Partitioning splits a large table into smaller physical pieces (partitions) while appearing as one logical table. This improves query performance and manageability for very large datasets (millions+ rows).

Partition Types

RANGE	-- Row goes to partition based on value range
LIST	-- Row goes to partition based on explicit value list
HASH	-- Evenly distributed across N partitions
KEY	-- Similar to HASH using MySQL's own function
COLUMNS	-- RANGE/LIST on multiple columns

RANGE Partitioning

```
CREATE TABLE orders (  
  order_id INT NOT NULL,  
  order_date DATE NOT NULL,  
  amount DECIMAL(10,2)  
)  
PARTITION BY RANGE (YEAR(order_date)) (  
  PARTITION p2020 VALUES LESS THAN (2021),  
  PARTITION p2021 VALUES LESS THAN (2022),  
  PARTITION p2022 VALUES LESS THAN (2023),  
  PARTITION p2023 VALUES LESS THAN (2024),  
  PARTITION p_max VALUES LESS THAN MAXVALUE  
)  
;
```

LIST Partitioning

```
CREATE TABLE employees (  
  emp_id INT NOT NULL,  
  name VARCHAR(100),  
  region VARCHAR(20)  
)  
PARTITION BY LIST COLUMNS(region) (  
  PARTITION p_north VALUES IN ('Delhi','UP','Punjab'),  
  PARTITION p_south VALUES IN ('Kerala','Tamil Nadu','Karnataka'),  
  PARTITION p_west VALUES IN ('Maharashtra','Gujarat')  
)  
;
```

HASH Partitioning

```
CREATE TABLE logs (  
  log_id INT NOT NULL,  
  message TEXT,  
  created_at DATETIME  
)  
PARTITION BY HASH(log_id)  
PARTITIONS 8;
```

Manage Partitions

```
-- Add partition (RANGE only)
ALTER TABLE orders ADD PARTITION (
  PARTITION p2024 VALUES LESS THAN (2025)
);

-- Drop partition (and its data!)
ALTER TABLE orders DROP PARTITION p2020;

-- Truncate partition
ALTER TABLE orders TRUNCATE PARTITION p2021;

-- Show partitions
SELECT PARTITION_NAME, TABLE_ROWS
FROM information_schema.PARTITIONS
WHERE TABLE_NAME = 'orders';

-- EXPLAIN to verify partition pruning
EXPLAIN SELECT * FROM orders WHERE YEAR(order_date) = 2023;
-- partitions column shows which partitions are accessed
```



TIP

Partition pruning happens automatically when queries filter on the partition key — this is the main performance benefit.

🔧 Practice Tasks	
Task 1	Create a RANGE partitioned sales table partitioned by year.
Task 2	Insert records and verify which partition they went to using information_schema.
Task 3	Add a new partition for the next year to your sales table.
Task 4	Drop the oldest partition (verify data is removed).
Task 5	Use EXPLAIN to verify partition pruning occurs on a date-filtered query.

Chapter 23 — Security

Database security involves protecting data from unauthorized access, SQL injection, and misuse. Best practices include least-privilege access, SSL encryption, password policies, and regular audits.

Password Policy

```
-- Check password policy
SHOW VARIABLES LIKE 'validate_password%';

-- Set password policy
SET GLOBAL validate_password.policy = MEDIUM;
-- LOW: Length only
-- MEDIUM: Length + numbers + uppercase + special
-- STRONG: + dictionary file

SET GLOBAL validate_password.length = 12;
SET GLOBAL validate_password.mixed_case_count = 1;
SET GLOBAL validate_password.special_char_count = 1;
```

Secure User Creation

```
-- Create user with strong password
CREATE USER 'appuser'@'192.168.1.%'
IDENTIFIED BY 'Str0ng#P@ssword!'
PASSWORD EXPIRE INTERVAL 90 DAY -- Expire every 90 days
FAILED_LOGIN_ATTEMPTS 5         -- Lock after 5 failures
PASSWORD_LOCK_TIME 2;           -- Lock for 2 days

-- Grant minimum needed privileges (principle of least privilege)
GRANT SELECT, INSERT, UPDATE ON app_db.* TO 'appuser'@'192.168.1.%';
```

SSL / TLS Connections

```
-- Check SSL status
SHOW VARIABLES LIKE '%ssl%';
SHOW STATUS LIKE 'Ssl_cipher';

-- Require SSL for a user
ALTER USER 'appuser'@'%' REQUIRE SSL;
ALTER USER 'appuser'@'%' REQUIRE X509; -- Require client cert

-- Connect with SSL from client
-- mysql -u user -p --ssl-ca=ca.pem --ssl-cert=client-cert.pem --ssl-key=client-key.pem
```

Preventing SQL Injection

```
-- ✗ VULNERABLE (never do this)
-- "SELECT * FROM users WHERE name = '" + user_input + '"
```

```
-- ☒ SAFE: Use Prepared Statements
PREPARE stmt FROM 'SELECT * FROM users WHERE name = ?';
SET @name = 'Alice';
EXECUTE stmt USING @name;
DEALLOCATE PREPARE stmt;

-- In application code: always use parameterized queries / ORM
```

Audit & Monitoring

```
-- Check who is connected
SHOW PROCESSLIST;
SELECT * FROM information_schema.PROCESSLIST;

-- Kill a connection
KILL CONNECTION 42;

-- Enable general query log (careful: logs everything)
SET GLOBAL general_log = 'ON';
SHOW VARIABLES LIKE 'general_log%';

-- Check failed login attempts (MySQL 8.0+)
SELECT user, host, Failed_login_attempts FROM mysql.user;
```

NOTE

Never run MySQL as root. Always use application-specific users with minimal required privileges.

Practice Tasks

Task 1	Enable validate_password plugin and set policy to MEDIUM.
Task 2	Create a user with password expiry of 60 days and 3 failed login lock.
Task 3	Require SSL for a specific user and verify the requirement.
Task 4	Demonstrate a prepared statement that prevents SQL injection.
Task 5	List all currently connected users and kill a test connection.

Chapter 24 — Backup & Restore

Backups protect your data from hardware failures, mistakes, and disasters. MySQL provides several backup tools. `mysqldump` is the most common for logical backups; `mysqlpump` and MySQL Enterprise Backup are alternatives.

mysqldump — Logical Backup

```
-- Backup entire server
mysqldump -u root -p --all-databases > all_dbs.sql

-- Backup single database
mysqldump -u root -p company > company_backup.sql

-- Backup specific tables
mysqldump -u root -p company employees orders > partial.sql

-- Backup with options
mysqldump -u root -p \
  --single-transaction \    -- Consistent backup without locking
  --routines \              -- Include stored procedures/functions
  --triggers \              -- Include triggers
  --events \                -- Include scheduled events
  company > company_full.sql

-- Compress backup
mysqldump -u root -p company | gzip > company_$(date +%Y%m%d).sql.gz
```

Restore from Backup

```
-- Restore entire backup
mysql -u root -p < all_dbs.sql

-- Restore to specific database
mysql -u root -p company < company_backup.sql

-- Restore compressed backup
gunzip < company_20240115.sql.gz | mysql -u root -p company

-- Via MySQL client
mysql> SOURCE /path/to/company_backup.sql;
```

Binary Log Backup (Point-in-Time Recovery)

```
-- Check if binary logging is enabled
SHOW VARIABLES LIKE 'log_bin';
SHOW BINARY LOGS;
SHOW MASTER STATUS;
```

```
-- View binary log events
SHOW BINLOG EVENTS IN 'mysql-bin.000001' LIMIT 20;

-- Replay binary log (recovery)
mysqlbinlog mysql-bin.000001 | mysql -u root -p

-- Recover to specific time
mysqlbinlog --start-datetime='2024-01-15 10:00:00' \
  --stop-datetime='2024-01-15 11:00:00' \
  mysql-bin.000001 | mysql -u root -p
```

Export / Import with SELECT INTO OUTFILE

```
-- Export to CSV
SELECT * FROM employees
INTO OUTFILE '/tmp/employees.csv'
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n';

-- Import CSV
LOAD DATA INFILE '/tmp/employees.csv'
INTO TABLE employees
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS; -- Skip header
```



TIP

Use --single-transaction for InnoDB tables to get a consistent backup without table locks.

 Practice Tasks	
Task 1	Backup the company database with routines and triggers included.
Task 2	Restore the backup to a new database called company_test.
Task 3	Export the employees table to a CSV file.
Task 4	Import a CSV file into a staging table.
Task 5	Check the binary log status and list binary log files.

Chapter 25 — Replication

MySQL Replication copies data from one server (Primary/Source) to one or more servers (Replicas). Used for read scaling, high availability, and backups. The primary logs changes to binary logs; replicas read and apply them.

Replication Architecture

Source (Primary)	Replica (Secondary)
-- Binary Log (binlog) -->	
	Relay Log
	IO Thread (reads binlog)
	SQL Thread (applies changes)
Types:	
Async -- Default. Replica may lag behind Primary	
Semi-sync -- Primary waits for at least 1 replica to ack	
Group Replication -- Multi-primary (InnoDB Cluster)	

Configure Source (Primary)

-- In my.cnf on source server:
[mysqld]
server-id = 1 # Unique ID for each server
log_bin = mysql-bin # Enable binary logging
binlog_format = ROW # ROW, STATEMENT, or MIXED
binlog_do_db = company # Replicate only this DB (optional)
-- Restart MySQL, then:
-- Create replication user
CREATE USER 'replicator'@'replica_ip'
IDENTIFIED WITH mysql_native_password BY 'Repl@Pass123';
GRANT REPLICATION SLAVE ON *.* TO 'replicator'@'replica_ip';
FLUSH PRIVILEGES;
-- Get binary log position
FLUSH TABLES WITH READ LOCK;
SHOW MASTER STATUS; -- Note File and Position values
UNLOCK TABLES;

Configure Replica

-- In my.cnf on replica:
[mysqld]
server-id = 2 # Must be different from source
relay_log = relay-bin
-- On replica MySQL:
CHANGE REPLICATION SOURCE TO

```

SOURCE_HOST = '192.168.1.10',
SOURCE_USER = 'replicator',
SOURCE_PASSWORD = 'Repl@Pass123',
SOURCE_LOG_FILE = 'mysql-bin.000001', -- From SHOW MASTER STATUS
SOURCE_LOG_POS = 154;                -- From SHOW MASTER STATUS

START REPLICATION;
SHOW REPLICATION STATUS\G

```

Monitor Replication

```

-- On replica
SHOW REPLICATION STATUS\G
-- Look for:
-- Replica_IO_Running: Yes
-- Replica_SQL_Running: Yes
-- Seconds_Behind_Source: 0 (lag)
-- Last_Error: (empty = good)

-- Stop / Start
STOP REPLICATION;
START REPLICATION;

-- Reset replica
STOP REPLICATION;
RESET REPLICATION ALL;

```

GTID-Based Replication (MySQL 5.7+)

```

-- In my.cnf (both servers):
gtid_mode = ON
enforce_gtid_consistency = ON

-- Setup is simpler — no need for log file/position:
CHANGE REPLICATION SOURCE TO
  SOURCE_HOST = '192.168.1.10',
  SOURCE_USER = 'replicator',
  SOURCE_PASSWORD = 'Repl@Pass123',
  SOURCE_AUTO_POSITION = 1; -- GTID handles position automatically

START REPLICATION;

```



TIP

GTID-based replication is recommended for new setups — easier failover and no need to track binary log file positions.

Practice Tasks

Task 1	Configure a source server with binary logging enabled and create a replication user.
Task 2	Configure a replica to connect to the source using CHANGE REPLICATION SOURCE TO.
Task 3	Start replication and verify both IO and SQL threads are running.
Task 4	Check Seconds_Behind_Source to monitor replica lag.

Task 5

Stop replication, reset the replica, and reconfigure with GTID-based replication.

